

A Product Model for Sources with Memory

Experimental log

July 28, 2026

Abstract

This document records the experiments of the `product_model_with_memory` project, which extends the layered product-simplex predictor of [1] from exchangeable (memoryless) sources to sources with memory. The past is summarized in a state, and the scheme predicts from (state, next token) pairs; the simplest instance is a first-order Markov model in which the state is the previous token. The document is a running log: sections are added as experiments complete, and every number is produced by a script in the repository.

1 Setup and notation

We use the layered product-simplex model of [1]. It is built in three steps: a prior on the simplex, a Bayesian mixture over sequences, and an average over depths.

The prior. Fix an alphabet of size d and a depth $L \geq 1$. Draw L points $\theta^{(1)}, \dots, \theta^{(L)}$ of the d -simplex independently and uniformly, and form the coordinatewise product, renormalized:

$$\theta_i = \frac{\prod_{\ell=1}^L \theta_i^{(\ell)}}{\sum_{j=1}^d \prod_{\ell=1}^L \theta_j^{(\ell)}}, \quad i = 1, \dots, d. \quad (1)$$

The law of the resulting random vector θ is the depth- L prior π_L on the simplex. At $L = 1$ it is the uniform prior; as L grows, the product concentrates the mass of π_L on increasingly sparse, low-entropy vectors. The depth thus parametrizes a family of priors ranging from “all d symbols comparably likely” to “a few symbols carry almost all the mass”.

The mixture over sequences. The data are sequences, not single observations. Given θ , a sequence $x^N = (x_1, \dots, x_N)$ of N tokens is drawn i.i.d. from θ : each x_t takes the value i with probability θ_i . The depth- L predictor is the Bayesian mixture

$$Q_N^{(L)}(x^N) = \int \prod_{t=1}^N \theta_{x_t} \pi_L(d\theta), \quad (2)$$

a normalized probability assignment on all d^N sequences of length N . Its conditionals $Q^{(L)}(x_{t+1} = i | x^t)$ are the posterior means $\mathbb{E}[\theta_i | x^t]$; at $L = 1$ this conditional is exactly Laplace’s add-one rule $(m_i + 1)/(t + d)$, with m_i the number of occurrences of i in x^t . Because the data are i.i.d. given θ , the mixture (2) is exchangeable: it depends on x^N only through the counts $m_1, \dots, m_d$.

The average over depths. The depth-averaged predictor

$$Q_N^{\text{avg}}(x^N) = \frac{1}{L_{\max}} \sum_{L=1}^{L_{\max}} Q_N^{(L)}(x^N), \quad L_{\max} = \text{round}(2c^* \ln d), \quad c^* = \frac{1}{1 - \gamma} \approx 2.365, \quad (3)$$

requires no depth selection.

The two quantities we report. Throughout, a corpus prefix $x^n = (x_1, \dots, x_n)$ is a token sequence over a fixed alphabet of size d , with counts $m_i = \#\{t : x_t = i\}$.

Empirical entropy. The empirical unigram entropy of the prefix is the entropy of its type (relative-frequency) distribution,

$$\hat{H}_n = - \sum_{i: m_i > 0} \frac{m_i}{n} \log_2 \frac{m_i}{n} \quad [\text{bits/token}]. \quad (4)$$

It is an in-sample quantity, and it is worth being precise about the sense in which it is a floor. An arithmetic code driven by a fixed probability assignment p on the alphabet spends $-\frac{1}{n} \sum_t \log_2 p(x_t)$ bits per token; over all constant (i.i.d.) assignments this is minimized at $p_i = m_i/n$, the relative frequencies of the prefix itself, and the minimum value is $\hat{H}_n$. So no constant assignment can do better than $\hat{H}_n$ on this prefix. But $\hat{H}_n$ is not itself the length of a code. Decoding an arithmetic code requires the decoder to use the same probabilities as the encoder, and while the encoder can compute m_i/n — it has the whole prefix in front of it — the decoder cannot: the relative frequencies are a function of exactly the data it has not yet received. A decodable scheme must close this gap in one of two ways. Either transmit the frequencies first and then code the data under them (a two-part code, which pays the description cost of the frequencies), or use at step t only probabilities computable from x^{t-1} , which encoder and decoder both know at that point (a sequential code, which pays instead through its early predictions, made from few observations). All codelengths reported in this paper are of the second kind. $\hat{H}_n$ is thus the floor against which any memoryless estimator is measured, and the excess above it is the cost of closing this gap.

Loss of an estimator. An estimator is a probability assignment Q on sequences (nonnegative, summing to one over $\mathcal{X}_d^n$). Its loss on the prefix is the per-token codelength

$$\ell_n(Q) = -\frac{1}{n} \log_2 Q(x^n) = -\frac{1}{n} \sum_{t=1}^n \log_2 Q(x_t | x^{t-1}), \quad (5)$$

the two forms being equal by the chain rule; by the Kraft inequality this is (up to $O(1/n)$) the length of an actual arithmetic code, so $\ell_n(Q)$ is realizable, not merely a score. The *redundancy* we tabulate is $\ell_n(Q) - \hat{H}_n$: the part of the codelength paid for *learning* the distribution rather than describing the data under a known one.

For the mixtures used here the sequential product in (5) never needs to be accumulated term by term: the mixture is exchangeable, so $Q(x^n)$ depends on x^n only through its count profile λ (the multiset of nonzero counts), and $-\log_2 Q^{\text{avg}}(x^n) = -\log_2 [\frac{1}{L_{\max}} \sum_L q_\lambda(L)]$, where $q_\lambda(L)$ is evaluated by the layer recursion and saddlepoint method of Appendix B of [1], ported to this repository (`src/product_model_with_memory/layered.py`, with the batched table builder of `fast_tables.py` validated against the original).

Depth posterior. Because (3) is a uniform mixture over depths, the data induce a posterior over L ,

$$w_L(x^n) = \frac{Q^{(L)}(x^n)}{\sum_{L'=1}^{L_{\max}} Q^{(L')}(x^n)}, \quad (6)$$

which we report through its mode $L^* = \arg \max_L w_L(x^n)$. The mode plays no role in any codelength — the loss is always that of the full average (3) — but it is an interpretable diagnostic: it names the single depth that best explains the data, and its position (relative to $\ln d$) locates the source on the complexity spectrum of [1].

2 Validation: reproducing the King James Bible experiment

Before any new experiment we re-run the real-text experiment of [1], Section 5.3 (Table 10) with the ported code. The paper used Project Gutenberg etext #10 (915,849 word-and-punctuation tokens, 13,550 distinct types); our corpus is the closely related eBook #30 (“The Bible, King James Version, Complete”), which after removal of the header, verse markers, and book headings tokenizes to 917,150 tokens and 13,542 types (a 0.15% edition difference). Alphabet $d = 100,000$, $L_{\max} = \text{round}(2c^* \ln d) = 54$. Table 1 compares the published redundancies of the depth-averaged predictor with our reproduction.

n	$\hat{H}_n$ (paper/ours)	redundancy PS avg		mode L	
		paper	ours	paper	ours
10,000	7.56 / 7.504	1.052	1.019	21	22
30,000	7.87 / 7.844	0.598	0.584	21	21
100,000	8.02 / 8.013	0.306	0.302	21	21
300,000	8.18 / 8.182	0.168	0.167	19	19
full text	8.48 / 8.478	0.095	0.095	15	15

Table 1: Reproduction of the King James Bible experiment ([1], Table 10) with the ported code (`scripts/unigram_experiment.py`) on eBook #30. Redundancy in bits/token of the depth-averaged product-simplex predictor. Agreement is to the third decimal on the full text and within the differences expected from the two editions at the smallest prefixes (at $n = 10^4$ the two corpora differ most in relative terms: 1,108 vs. 1,161 observed types); the posterior depth sequence $21 - 22 \rightarrow 15$ is reproduced. We regard the port as validated.

3 The text8 corpus: memoryless baseline

The first corpus beyond the Bible is **text8**: the first 10^8 bytes of an English Wikipedia dump, stripped to lowercase a–z and space (17,005,207 tokens, 253,854 distinct types; canonical file, md5 `3bea1919949baf155f99411df5fada7e`). Empirical unigram entropy of the full corpus: 10.856 bits/token; empirical conditional entropy given the previous token: 7.286 bits/token, so first-order memory is worth about 3.57 bits/token in sample. We take $d = 2^{18} = 262,144$ (the smallest power of two above the number of observed types), giving $L_{\max} = 59$.

The column “mode L ” in Table 2 is the mode L^* of the depth posterior (6): among the $L_{\max} = 59$ single-depth mixtures $Q^{(L)}$ that make up the average (3), it is the depth to which the data assign the largest posterior weight $w_L \propto Q^{(L)}(x^n)$ — equivalently, the depth whose mixture gives the prefix the shortest codelength. It is a reported quantity, not a choice: every codelength in the table is that of the full depth average, in which all 59 depths participate with their posterior weights. Watching L^* move is nonetheless informative, and Table 2 shows a pronounced drift discussed in the caption.

4 Pairs: plug-in conditionals from a joint mixture

The first memory experiment applies the same machinery to sliding-window pairs. We describe exactly what `scripts/pair_experiment.py` computes.

n	types	$\hat{H}_n$	PS avg	redundancy	mode L
10,000	2,520	9.275	11.462	2.187	15
30,000	5,651	9.736	11.183	1.446	13
100,000	12,023	10.109	10.939	0.830	12
300,000	24,537	10.419	10.911	0.492	10
1,000,000	52,754	10.614	10.876	0.262	8
3,000,000	96,644	10.719	10.853	0.134	6
10,000,000	189,074	10.852	10.910	0.058	4
17,005,207	253,854	10.856	10.895	0.039	4

Table 2: Memoryless (unigram) baseline on `text8` ($d = 2^{18}$, $L_{\max} = 59$; full run, 20 cores, 41 min): codelength per token of the depth-averaged predictor, redundancy above the empirical unigram entropy of the prefix, and the posterior-mode depth. Two observations. First, the redundancy decays smoothly to 0.039 bits/token on the full corpus. Second, the selected depth falls from $L = 15$ ($\approx 1.2 \ln d$) to $L = 4$ ($\approx 0.3 \ln d$): unlike the Bible (13.5% of a 10^5 vocabulary used, depth settling at $1.3 \ln d$), `text8` eventually reveals 97% of its 2^{18} -sized alphabet, the target is not sparse relative to d , and the posterior moves toward the shallow (Laplace-like) end. This is the behaviour [1] predicts for targets that are not sparse relative to the alphabet, observed here on one corpus by varying n alone.

Vocabulary sequence. To scale from laptop to cluster we reduce the vocabulary to the K most frequent tokens plus a single `<unk>` bucket ($V = K + 1$ symbols; the structure of the order-one partition of [1], Section 5.4). All quantities below refer to the *reduced* token stream $x_1, \dots, x_n$; the pair alphabet is $d = V^2$, and $L_{\max} = \text{round}(2c^* \ln d)$ as always. The targets are the empirical entropies of the same reduced stream: the unigram entropy $\hat{H}_1$ and the conditional entropy $\hat{H}(x_{t+1} | x_t) = \hat{H}_{\text{pair}} - \hat{H}_{\text{first}}$, both in-sample.

Estimate. The joint distribution of pairs is estimated *in-sample* (plug-in): the entire multiset of $n - 1$ sliding pairs, with count profile λ , informs the estimate, and the log-loss is evaluated on the same sequence. The estimate is the depth-averaged mixture’s posterior mean. For a pair currently at count c (including $c = 0$ for unobserved pairs),

$$\hat{p}(c) = \sum_{L=1}^{L_{\max}} w_L p_L(c), \quad w_L = \frac{q_L(\lambda)}{\sum_{L'} q_{L'}(\lambda)}, \quad p_L(c) = \frac{q_L(\lambda + e_c)}{q_L(\lambda)}, \quad (7)$$

where $\lambda + e_c$ moves one symbol from count class c to $c + 1$. The ratio $p_L(c)$ is not evaluated by two independent profile computations; at a saddle u^* of the outer integral the posterior mean of a coordinate with count c is

$$p_L(c) \approx \frac{\rho_L(c, u^*)}{N}, \quad \rho_L(c, u) = e^u \frac{\phi_{c+1}^{(L)}(e^u)}{\phi_c^{(L)}(e^u)},$$

with $\phi_r^{(L)}$ the layer-recursion moment functions of [1], Appendix B. The saddle equation $\sum_i \rho_L(c_i, u^*) = N$ makes the class-weighted sum of these predictive masses normalize *exactly* at saddle order; when the integrand has several significant peaks (deep layers with heavy counts) the peaks are combined with their scan weights. $p_L(1)$ is exact add-one at $L = 1$. A final numerical renormalization absorbs the residual $O(1/N)$ -level error, which the program reports as a diagnostic (validated against exact

ratio computations to $< 2\%$ per depth in the test suite; the aggregate normalization error is far smaller).

Conditionals and score. Conditional probabilities come from row normalization, $\hat{p}(b|a) = \hat{p}(c_{ab}) / \sum_{b'} \hat{p}(c_{ab'})$, with unobserved successors carrying the class-0 mass; the row sums reduce to per-row count histograms. The score is the in-sample conditional log-loss

$$\ell = -\frac{1}{n-1} \sum_{t=2}^n \log_2 \hat{p}(x_t | x_{t-1}) = -\frac{1}{n-1} \sum_{a,b} m_{ab} \log_2 \hat{p}(b|a),$$

compared like-for-like with the in-sample $\hat{H}(x_{t+1}|x_t)$; the gap $\ell - \hat{H}(x_{t+1}|x_t)$ is the estimator’s smoothing-plus-discovery cost. This plug-in scheme is a diagnostic, not a code: fully sequential (decodable) variants — the per-state construction of [1] Section 5.4, and the causal joint-conditional predictor with checkpointed refreshes — are deferred to a later stage.

K	$d = V^2$	$L_{\max}$	pairs obs.	$\hat{H}_1$	$\hat{H}(x' x)$	ℓ (plug-in)	gap
63	2^{12}	39	3,762	3.224	2.733	2.7334	0.00001
255	2^{16}	52	45,076	4.423	3.709	3.7098	0.00055
1,023	2^{20}	66	307,076	6.063	4.979	4.9878	0.00850
4,095	2^{24}	79	1,172,941	8.051	6.338	6.3951	0.05747
16,383	2^{28}	92	2,578,046	9.702	7.154	7.3104	0.15624

Table 3: Pair experiment on full `text8` ($n = 17,005,207$ tokens, $n - 1$ sliding pairs), vocabulary reduced to the top K tokens plus $\langle \text{unk} \rangle$. Empirical (in-sample) unigram and conditional entropies of the reduced stream in bits/token, the plug-in conditional log-loss ℓ of the depth-averaged joint estimate, and the gap $\ell - \hat{H}(x'|x)$. At the first two rungs the pair alphabet is densely observed ($\approx 4,150$ and 260 samples per possible pair) and the plug-in estimate sits essentially on the ceiling, with the joint posterior a point mass at $L = 7$ resp. $L = 6$ (≈ 0.5 – $0.8 \ln d$); the sparsely observed regime begins at $K = 1,023$ (≤ 16 samples per possible pair, 71% of the pair alphabet unseen), where the gap opens but remains small: 0.0085 bits/token, i.e. 0.17% of the conditional entropy, with the joint posterior again a point mass ($L = 8$). The sequence of gaps ($1.3 \cdot 10^{-5} \rightarrow 5.5 \cdot 10^{-4} \rightarrow 8.5 \cdot 10^{-3} \rightarrow 5.7 \cdot 10^{-2} \rightarrow 1.56 \cdot 10^{-1}$) measures the discovery cost that made the earlier full-vocabulary attempt impractical; its growth factors from one row to the next decrease ($\times 42 \rightarrow \times 15 \rightarrow \times 6.8 \rightarrow \times 2.7$). Even at $K = 16,383$, with 99% of the 2^{28} pair alphabet unseen and 6.6 samples per observed pair, the gap is 2.2% of the conditional entropy. The joint posterior remains a point mass throughout, at depths $L = 7, 6, 8, 10, 15$ that grow as the pair distribution becomes sparse relative to its alphabet, as in Table 2. Predictive normalization diagnostics: 10^{-13} – 10^{-6} in all rows.

5 Program: averaging over a family of state maps

The pair experiment is the first step of a more general program, which we record here because it fixes the design principle for everything that follows.

A memory model is a *state map* σ compressing the past into a state, $s_t = \sigma(x^{t-1})$, followed by the estimation machinery on (state, next token) pairs. The quality of σ at a given data budget decomposes into two terms that move in opposite directions:

- the *ceiling* $\hat{H}(X_{t+1} | S)$ — the best conditional log-loss any predictor built on these states can reach — which falls as states become finer (last token, last two tokens, ...); and
- the *learning cost* — the redundancy paid to estimate one conditional per state — which grows with the number of states and is governed, as in the scaling analysis of [1], by the rate at which new (state, successor) pairs are discovered.

Total codelength is the sum, and “finding good states” means optimizing this sum. Both terms are computable in our framework: the ceiling empirically, the learning cost exactly through the regret calculus. Section 5.4 of [1] exhibits the trade-off in miniature (the interior optimum in the number of split contexts); here it becomes the main object of study.

The design principle is to treat memory *exactly as depth is treated*. The depth-averaged predictor does not select a depth: it averages the mixtures of all depths $L \leq L_{\max}$ and pays at most $(\log_2 L_{\max})/n$ per token for not knowing the right depth in advance, a price that vanishes as n grows. Correspondingly, we do not intend to select a single state map: we define a *nested family* $\sigma_1 \preceq \sigma_2 \preceq \dots \preceq \sigma_M$ of state maps of increasing memory (each refining the last), place a prior over the family, and use the single coherent Bayesian mixture

$$Q(x^n) = \sum_{m=1}^M \pi_m Q^{(\sigma_m)}(x^n), \quad (8)$$

where $Q^{(\sigma_m)}$ is the (depth-averaged) layered predictor built on the states of σ_m . The data then *selects the effective memory* through the posterior over m , just as it now selects the effective depth, at a model-selection price of at most $(\log_2 M)/n$ bits per token. Memory is then handled in the same way as depth: not one σ for all sources, but one prior whose mass is spread over all the state maps the family contains.

Concrete instances of the family, in increasing order of generality: (i) the top- K sequence of Section 4, i.e. the last token bucketed by frequency, a first coarse compression of the past into a state; (ii) variable-length suffixes of the past, i.e. context trees, where averaging over the family recovers the structure of context-tree weighting but with layered mixtures at the leaves in place of KT estimators (the layered prior dominates KT on large alphabets [1]); (iii) states obtained by *clustering* last tokens by similarity of their successor distributions (merge states that predict alike, sweep the cluster count), the first family whose members are constructed from the data rather than fixed in advance; (iv) learned continuous state maps, which connect this program to the architecture-as-prior view of [1], Section 6: a state map is an architecture, and its design determines where the prior puts its mass over predictors. The limiting case — states defined by “two pasts are equivalent iff they induce the same conditional law of the future” (causal states) — cannot be learned directly; it describes what the constructions above approximate.

Orthogonal to the choice of states is the question of *coupling*: whether the per-state conditionals are estimated independently (the share-nothing construction of [1], Section 5.4) or jointly, as in Section 4, where the prior on the joint pair distribution lets every row’s smoothing be informed by the count statistics of all rows. The two axes — what the states are, and whether they share statistical strength — should be varied separately; the comparison of joint-coupled versus share-nothing estimation at identical states isolates the value of coupling.

5.1 First instance: the context-partition family

The simplest implementation of the program is the following (`scripts/state_family_experiment.py`). Fix the emission vocabulary to the top K tokens plus $\langle \text{unk} \rangle$ ($V = K+1$). The family members are

the state maps

$$\sigma_M : \quad s_t = \begin{cases} x_{t-1} & \text{if } x_{t-1} \text{ is among the top } M \text{ tokens,} \\ \langle \text{other} \rangle & \text{else,} \end{cases} \quad M \in \{0, 1, 2, 4, \dots, V\},$$

nested and log-spaced; σ_0 has a single state and *is* the memoryless predictor of Section 3, so the family contains “no memory” as its coarsest member. Each member is scored by the sequential share-nothing code: conditioned on the state the successor sub-sequence is exchangeable, so $Q^{(\sigma_M)}(x^n) = \prod_s q^{\text{avg}}(\text{successor profile of } s)$, every factor a per-state depth-averaged layered mixture evaluated exactly from counts (this is the construction of [1], Section 5.4, rebuilt from the validated port). The family average is the uniform mixture (8); all members share one emission alphabet and hence one moment-table cache, so the whole family costs little more than its finest member.

Three predictions are checked. (i) The member codelengths trace an interior optimum in M : coarse members pay in ceiling, fine members pay in learning cost, and the minimizer is the data-determined effective memory. (ii) The posterior over M concentrates at that optimum. (iii) The family mixture tracks the best member to within $(\log_2 |\text{grid}|)/n$ bits/token, so memory-averaging, like depth-averaging, costs almost nothing. The member codelengths are those of sequential codes; comparing the best member with the in-sample plug-in value of Table 3 measures the additional cost of estimating the conditionals as the data arrive rather than from the whole corpus.

Order-two extension (implemented). The family need not stop at the previous token. With the same fixed emission vocabulary, define product states with *asymmetric resolution*,

$$\sigma_{M_1, M_2} : \quad s_t = (b_{M_1}(x_{t-1}), b_{M_2}(x_{t-2})),$$

where b_M keeps a token if among the top M and buckets it otherwise (b_0 buckets everything). The grid over (M_1, M_2) contains the first-order family as its $M_2 = 0$ slice and memoryless as $(0, 0)$; all members code $x_3, \dots, x_n$ over the same alphabet and are therefore mixable exactly as before (`scripts/product_family_experiment.py`). The posterior over the grid is now a measurement of how predictive resolution should be *split across distances into the past* at a given data budget. Computationally the member count is reduced by a global deduplication: rare contexts overwhelmingly share identical tiny successor profiles, so the $\sim 2 \cdot 10^5$ states of a ten-member grid on full `text8` collapse to $\sim 2.7 \cdot 10^4$ unique profiles, each evaluated once.

The first order-two experiment

The first run of this family (full `text8`, $V = 1,024$) uses the ten-member grid

$$\{(0, 0)\} \cup \{(M, 0) : M = 64, 256, 1024\} \cup \{(1024, M_2) : M_2 = 4, 16, 64, 256\} \cup \{(256, 16), (64, 64)\},$$

designed to ask three questions at once. (i) The $M_2 = 0$ slice anchors the grid to the first-order family of Table 6 (values shift microscopically because all members now code $x_3, \dots, x_n$). (ii) The arm $(1024, M_2)$ with growing M_2 asks: *on top of full one-back resolution, what is progressively finer resolution of the token two positions back worth, and when does it stop paying?* This is the second axis of the boundary-versus-interior question that the first-order family, monotone up to $M = V$, left open. (iii) The members $(256, 16)$ and $(64, 64)$ hold the number of states roughly comparable to $(1024, 0)$ ($\sim 1\text{--}4\text{k}$ states) while splitting resolution differently across the two distances: *given a budget of states, should they resolve the recent past finely, or spread across more history?* The

posterior over the grid answers all three; the outcomes of interest are whether the second axis, too, is monotone up to the boundary at this n (pushing the effective-memory frontier beyond order one entirely), and, if not, where the interior split lands.

M_1	M_2	states obs.	bits/token	posterior
0	0	1	6.0636	0
64	0	65	5.3773	0
256	0	257	5.2144	0
1024	0	1,024	5.0564	0
1024	4	4,989	5.0395	0
1024	16	15,950	5.0138	0
1024	64	47,687	5.0104	1.0
1024	256	127,078	5.0419	0
256	16	4,251	5.1411	0
64	64	3,891	5.2443	0

Table 4: Order-two product-state family on full `text8` ($V = 1,024$, sequential per-state codes, all members coding $x_3, \dots, x_n$; family mixture 5.0104 bits/token = best member). Along the arm $(1024, M_2)$ the codelength decreases through $M_2 = 4, 16$ to its minimum at $M_2^* = 64$ and increases again at $M_2 = 256$: resolution 64 on the token two positions back reduces the codelength by 0.046 bits/token relative to $(1024, 0)$, while finer two-back resolution costs more to learn than it gains. This is the first family in the sequence whose optimum lies in the interior of the grid, and the posterior concentrates on it. The constant-budget members answer the split question: at roughly equal state counts, $(1024, 4)$ gives a shorter codelength than $(256, 16)$, which gives a shorter codelength than $(64, 64)$; resolution on the most recent token is worth more than resolution further back. Along the first-order axis (Table 6) the optimum was always at the boundary; the trade-off appears once memory extends further back.

Scaling sequence. As with the pair experiment, the same code is run at a sequence of scales: (1) $V = 64$, $n = 10^6$ (small test run); (2) $V = 256$, full `text8` (laptop); (3) $V = 1,024$ and $V = 4,096$, full corpus (laptop, overnight), where the fine members are sparsely observed; (4) V at the full vocabulary and richer families (order-two states on the bucketed alphabet, clustered states), on the cluster. The quantities tracked are the location M^* of the optimum as a function of (V, n) , and the margin between the family mixture and both its $M = 0$ member (the value of memory) and the in-sample plug-in value of Table 3 (the cost of sequential estimation).

5.2 Adaptive depth: the context-tree family

The return of the optimum to the boundary in Table 5 reflects a restriction of the product family rather than a property of second-order memory itself: σ_{M_1, M_2} applies the same two-back resolution after every first-order state, frequent or rare. The restriction is removed by a family whose members allocate depth *per context*; the classical such family is the set of variable-length suffix state maps: prunings of the depth- D suffix tree, where a pruning maps the past x^{t-1} to its longest suffix that is a leaf. A pruning may look two tokens back after “of” and zero tokens back after a rare word, a freedom the product grid does not have (`scripts/context_tree_experiment.py`).

M_1	M_2	states obs.	bits/token	posterior
0	0	1	8.0523	0
256	0	257	7.0742	0
1024	0	1,025	6.8776	0
4096	0	4,096	6.7132	1.0
4096	16	56,096	6.7855	0
4096	64	147,784	6.8592	0
4096	256	318,201	6.9601	0
4096	1024	640,407	7.1486	0
1024	64	47,752	6.8777	0
1024	256	127,335	6.9317	0

Table 5: The same family at $V = 4,096$ (family mixture 6.7132 bits/token = best member): the optimum returns to the first-order boundary. Every allocation of two-back resolution now increases the codelength — (4096, 16) already costs 0.07 bits/token — and the two-back resolution that was worth 0.046 bits/token at $V = 1,024$ is worth nothing here: (1024, 64) equals (1024, 0) to four decimals. Together with Table 4 this shows that the location of the optimum is not monotone in V at fixed n : enlarging the emission alphabet raises every state’s estimation cost, and the data budget that at $V = 1,024$ left a surplus for order-two structure is at $V = 4,096$ fully consumed by order one. The result also shows a limitation of *uniform* product states: the same two-back resolution is applied after every first-order state, frequent or rare. A family whose members can allocate depth per context is the subject of Section 5.2.

Prior and mixture. The prunings cannot be enumerated (about 2^V of them already at depth two); instead the prior is generated recursively: starting from the empty context, each context splits into its per-token refinements with probability $\frac{1}{2}$, independently, up to maximal depth D . This is the first implemented instance of the principle of Section 7: the prior over state maps is defined by a random growth process rather than by enumeration. For a fixed pruning the predictor is the sequential share-nothing construction, one depth-averaged layered mixture per leaf, evaluated exactly from the leaf’s successor count profile; every member codes $x_{D+1}, \dots, x_n$ (the tokens with a full depth- D history), so all members are mixable and comparable. The mixture over *all* prunings then collapses, without enumeration, to the context-tree recursion

$$\beta(v) = \frac{1}{2} q^{\text{avg}}(v) + \frac{1}{2} \prod_{c \in \text{children}(v)} \beta(c), \quad (9)$$

evaluated bottom-up over the observed contexts, with $\beta = q^{\text{avg}}$ at depth D ; the family codelength is $-\log_2 \beta(\text{root})$. This is context-tree weighting with the layered mixture in place of the KT estimator at every node; the layered prior dominates KT on large alphabets [1]. Computationally the family costs one q^{avg} evaluation per distinct observed context of depth $\leq D$, with the same global profile deduplication as before; on full `text8` at $V = 1,024$, $D = 2$, the 308,101 observed contexts collapse to 32,479 unique profiles.

Reported quantities. Three numbers are reported. (i) The *fixed-depth baselines*: the complete tree at depth d , for $d = 0, 1, 2$. Depth 0 is the memoryless code and depth 1 the first-order share-nothing code, so these tie the run to Tables 6 and 4. (ii) The *family codelength* (9), which mixes over every pruning; since each complete tree is one pruning, the family is within its prior cost of

V	n	$\hat{H}(x' x)$	M^*	best member	family	$M = 0$ member
64	10^6	2.753	64 (bd.)	2.7644	2.7644	3.2573
256	$1.7 \cdot 10^7$	3.709	256 (bd.)	3.7198	3.7198	4.4234
1,024	$1.7 \cdot 10^7$	4.979	1024 (bd.)	5.0564	5.0564	6.0636
4,096	$1.7 \cdot 10^7$	6.338	4096 (bd.)	6.7132	6.7132	8.0523

Table 6: Context-partition family: location M^* of the posterior mode, best-member and family-mixture codelengths (bits/token, sequential codes), and the memoryless member, per scale (V, n) . At all scales run so far the optimum sits at the *boundary* (bd.): the member curve falls monotonically in M , the posterior puts mass 1.0 on the finest member, and the family mixture equals the best member to four decimals, as the $(\log_2 |\text{grid}|)/n$ bound requires. The boundary optimum persists even at $V = 1,024$, where 71% of the pair alphabet is unseen: a thousand share-nothing states are still worth their learning cost at $n = 1.7 \cdot 10^7$; with the layered per-state prior, the cost of adding rare states is small enough that refining pays at every scale tried. The sequential code obtains 98.6% ($V = 256$), 92.9% ($V = 1,024$), and 78.2% ($V = 4,096$) of the in-sample memory value $\hat{H}_1 - \hat{H}(x'|x)$; its excess over the plug-in value of Table 3 grows $0.0105 \rightarrow 0.0771 \rightarrow 0.3756$ bits/token. An interior optimum still does not appear: even four thousand share-nothing states, the rarest observed only dozens of times, each reduce the codelength by more than their learning cost at $n = 1.7 \cdot 10^7$. For last-token state maps at this corpus size the optimum lies at or beyond the full vocabulary; an interior optimum first appears when the family is extended to order two (Table 4).

the best fixed depth, and improves on it when per-context depth allocation reduces the codelength — the margin measures the value of that allocation. (iii) The *MAP pruning* (replace the sum in (9) by a max and keep back-pointers): its leaves-by-depth histogram reports *where* in the corpus depth is used, in addition to how much it is worth.

First run: full text8, $V = 1,024$, $D = 2$

Table 7 reports the run (n coded = 17,005,205, $L_{\max} = 33$; 31 minutes on 20 cores). The fixed-depth baselines land exactly on their counterparts in Tables 6 and 4 — depth 1 reproduces (1024, 0) to all reported decimals — validating the coding conventions across experiments.

Second run: full text8, $V = 4,096$, $D = 2$

The question left open by Table 5 was whether the return of the optimum to the first-order boundary at $V = 4,096$ meant that second-order structure had become worthless at that scale, or merely invisible to a uniform family. Table 8 shows the latter. The run ($L_{\max} = 39$; 1,177,038 observed contexts collapsing to 59,002 unique profiles; 46 minutes on 20 cores) again lands its depth-1 baseline exactly on the product family’s (4096, 0) member.

A remaining experiment is $D = 3$ at $V = 1,024$: whether any context supports a third token of depth.

5.3 The distance profile: the lag family

Before any pooling of lag experts (Section 7, scheme 5) can be designed, one should know where in the past the information sits. The lag family measures exactly this (`scripts/lag_family_experiment.py`). Its members are the single-lag state maps $\sigma^{(\delta)} : s_t = x_{t-\delta}$ at full vocabulary resolution, $\delta \in$

predictor	leaves / states	bits/token
complete depth 0 (memoryless)	1	6.0636
complete depth 1 (first order)	1,024	5.0564
complete depth 2	307,076	5.1309
best uniform product (1024, 64) [Tab. 4]	47,687	5.0104
context-tree family		4.9507
MAP pruning (967 leaves at depth 1, 26,506 at depth 2)	27,473	4.9507
first-order plug-in ceiling $\hat{H}(x' x)$ [Tab. 3]		4.9793

Table 7: Context-tree family with layered leaves on full `text8` ($V = 1,024$, $D = 2$). Three observations. (i) The complete depth-2 tree is worse than depth 1 by 0.074 bits/token — uniform two-back resolution increases the codelength, as in the product family — while the family, whose members split only where the data support it, is better than the best fixed depth by 0.106 and better than the best product member by 0.060 bits/token, with fewer states (27,473 leaves against 47,687). (ii) The family codelength 4.9507 is below $\hat{H}(x'|x) = 4.9793$, which is a lower bound for every predictor that conditions only on the previous token, including one given the in-sample conditional frequencies. The family is a sequential code, so the difference is produced by second-order structure. (iii) The MAP pruning splits 57 of the 1,024 depth-1 contexts (the other 967 remain leaves) into 26,506 depth-2 leaves, and the posterior puts mass ≈ 0.96 on this single pruning (MAP and family codelengths agree to $4 \cdot 10^{-9}$ bits/token, 0.06 bits in total). Second-order memory is used after 57 of the 1,024 possible previous tokens and nowhere else; a uniform (M_1, M_2) grid cannot represent such an allocation.

$\{1, 2, 3, 4, 6, 8\}$, plus the memoryless member ($\delta = 0$, a single state); all members code $x_{\delta_{\max}+1}, \dots, x_n$ and are mixed uniformly, so the posterior over δ locates the most useful lag. Two properties of the design are used below. First, every non-trivial member has exactly the same state space (V states, each a token identity), so the per-member learning costs are nearly identical and *differences* of member codelengths are essentially differences of ceilings. Relative to the one-state memoryless member, however, all members share a common learning-cost *offset* L (the cost of estimating V conditional distributions instead of one), so $v(\delta)$ below is the *net* value of lag δ , and estimates the raw pairwise information only up to L — a distinction invisible at $V = 1,024$ on short lags but, as the second run shows, both visible and measurable at $V = 4,096$. Second, the $\delta = 1$ member must reproduce the first-order family’s finest member, which it does (5.0564, matching Tables 6 and 4 to all reported decimals).

Scale two: $V = 4,096$, lags to 32. The second run (deltas $\{0, 1, 2, 3, 4, 6, 8, 12, 16, 24, 32\}$; 40,961 profiles, no deduplication benefit at full resolution; 85 minutes on 20 cores) is reported in Table 10. Its qualitatively new feature is that the *net* profile crosses zero: beyond $\delta \approx 4$, members are *worse* than memoryless: at this vocabulary a distant lag’s pairwise information no longer covers the cost of learning 4,096 conditional distributions. The crossing provides a measurement: as $\delta \rightarrow \infty$ the information vanishes and $v(\delta) \rightarrow -L$, so the tail level reveals the learning-cost offset directly, and the raw information profile is recovered as $I(\delta) = v(\delta) + \hat{L}$.

Lags to 64 at $V = 1,024$. The tail extension (deltas $\{0, 1, 2, 4, 8, 12, 16, 24, 32, 48, 64\}$; 63 minutes) is reported in Table 11. As at $V = 4,096$, the net profile crosses zero — here at $\delta \approx 24$ — and the tail level again measures the learning-cost offset, now $\hat{L} = 0.0153 \pm 0.0052$: nine times smaller than

predictor	leaves / states	bits/token
complete depth 0 (memoryless)	1	8.0523
complete depth 1 = (4096, 0) [Tab. 5]	4,096	6.7132
complete depth 2	1,172,941	7.4231
context-tree family		6.6807
MAP pruning (4,072 leaves at depth 1, 31,626 at depth 2)	35,698	6.6807
first-order plug-in ceiling $\hat{H}(x' x)$		6.3376

Table 8: Context-tree family on full `text8` ($V = 4,096$, $D = 2$). (i) At this scale every uniform allocation of two-back resolution increased the codelength (Table 5); the tree is nevertheless better than the best fixed depth — equivalently, the best product member — by 0.032 bits/token, obtained by splitting 24 of the 4,096 first-order contexts. The return of the optimum to the boundary in Table 5 was therefore a property of the uniform family, not an absence of second-order structure. (ii) Compared with $V = 1,024$ (Table 7), the gain over the best fixed depth falls from 0.106 to 0.032 bits/token and the number of split contexts from 57 to 24, while the penalty of the complete depth-2 tree grows to 0.710; the family codelength is no longer below $\hat{H}(x'|x)$ (6.6807 against 6.3376). As V grows at fixed n , second-order memory does not disappear; it is confined to fewer contexts. (iii) The posterior again concentrates, mass ≈ 0.93 on the single MAP pruning.

at $V = 4,096$ for a four-fold smaller state space.

The decay law: two regimes. With the tail measured, a single power law no longer fits: Figure 1 shows that both corrected profiles *break* at $\delta \approx 4$ into two power-law regimes (`scripts/plot_lag_profile.py`; the offset law $v = c\delta^{-\alpha} - L$ is fit on $\delta \geq 4$, determining $\hat{L}$ and the tail exponent, and a second power law is fit to the corrected short range $2 \leq \delta \leq 4$):

$$V = 1,024 : \alpha_s = 2.10, \alpha_t = 1.06 \pm 0.19, \hat{L} = 0.015 \quad V = 4,096 : \alpha_s = 2.02, \alpha_t = 1.34 \pm 0.20, \hat{L} = 0.137,$$

with tail RMS residuals of 0.004 bits at both scales (the best exponential-with-offset misses by $\sim 1.8\times$). The regime structure is the same at both scales — a steep short range with $\alpha_s \approx 2$, and a flat tail with α_t compatible between the scales, ≈ 1.0 – 1.3 — while V changes only the prefactors and the offset. A plausible interpretation, which we do not test here, is that the short-range regime reflects local sentence structure and the tail reflects slowly varying topic; published long-range exponents for natural language are in the range of our tail values. One consequence matters for what follows: α_t is close to 1, and the cumulative tail information $\sum_{\delta \leq \Delta} I(\delta) \sim \Delta^{1-\alpha_t}/(1-\alpha_t)$ (or $\log \Delta$ at $\alpha_t = 1$) keeps growing with the horizon Δ . No single far lag repays its own learning cost ($v < 0$ beyond $\delta \approx 24$ at $V = 1,024$ and beyond $\delta \approx 4$ at $V = 4,096$), but their total does not level off. A model that spends V conditional distributions per lag therefore cannot use the tail; a rule that combines per-lag predictions pays one weight per added lag (Section 7, scheme 5). How much of the total remains after accounting for the overlap between lags has not been measured.

6 Comparison with published results on `text8`

Units. Every number in this paper is in bits per *token*, computed by a single sequential pass over the whole corpus. The compression literature reports bits per *character* (bpc) on the raw file. `text8` contains 100,000,000 characters and 17,005,207 tokens, so one token accounts on average for

δ	bits/token	$v(\delta)$ (bits)	posterior
0 (memoryless)	6.0636	—	0
1	5.0564	1.0072	1.0
2	5.6962	0.3675	0
3	5.9046	0.1590	0
4	5.9902	0.0735	0
6	6.0293	0.0343	0
8	6.0418	0.0218	0
family mixture	5.0564		

Table 9: Lag family on full `text8` ($V = 1,024$; sequential share-nothing members, all coding $x_9, \dots, x_n$; 47 minutes on 20 cores). $v(\delta)$ is the member’s gain over the memoryless member: the value of the token δ positions back, as a function of δ . (i) The decay slows with distance: the factor from one lag to the next falls from $\times 2.7$ (lag 1 to 2) through $\times 2.3$ and $\times 2.2$ to about $\times 1.25$ per step at lags 6–8, and the token eight positions back still carries 0.022 bits/token. Decay slower than exponential agrees with published measurements of long-range correlations in natural language. (ii) The posterior puts mass 1.0 on $\delta = 1$ and the family equals its best member: a mixture selects one member and cannot combine several. The 0.656 bits/token in lags 2–8 combined is therefore not accessible to any mixture; using it requires combining predictions (Section 7, scheme 5). (iii) Lag 2 alone is worth 0.3675 bits, but the context tree at the same V (Table 7), which conditions on lags 1 and 2 jointly, gains only 0.106 over lag 1: most of the lag-2 information is also contained in lag 1. Any rule that combines per-lag predictions must account for this overlap.

5.8806 characters (its letters plus the following space). Dividing bits per token by 5.8806 therefore converts our numbers to bits per character. The conversion alone does not make the numbers comparable, for the reason spelled out next.

What our codes encode and what they leave out. All experiments in this paper run on a *reduced* stream: every token outside the K most frequent is replaced by the single symbol `<unk>`. Our codelength is the number of bits needed to transmit this reduced sequence. A decoder receiving it recovers the exact sequence of frequent words, but at each position holding an `<unk>` it learns only *that* a rare word occurred there — not which one. The original file cannot be reconstructed from what we transmit. At $V = 1,024$ this affects 32.5% of all tokens (average length 8.14 characters including the trailing space); at $V = 4,096$, 18.0% (8.37 characters). The published figures below, in contrast, are for programs whose output reconstructs the file byte for byte. To compare, we must additionally encode the spelling of every rare word. Because `text8` is exactly its tokens joined by single spaces, a token-level code plus a character-level code for the rare words reconstructs the file exactly; the cost of the character-level part is estimated below.

Published reference points and their rules. (i) *Mahoney’s Large Text Compression Benchmark* ranks programs on `enwik8` and `enwik9`, the raw Wikipedia dumps; `text8` is the cleaned, lower-cased first 10^8 bytes of `enwik9` and was created by Mahoney as the small test file for this benchmark. The ranking criterion is the compressed size *plus the size of the zipped decompressor*; the program makes one pass over the file, and everything the decoder needs is counted; time and memory are reported but not limited. This matches our setting: our decoder needs the algorithm and a handful of parameters, of negligible size. Mahoney’s test-data page lists results on `text8`

δ	bits/token	$v(\delta)$	$I(\delta) = v(\delta) + \hat{L}$
0 (memoryless)	8.0523	—	—
1	6.7132	1.3391	1.4759
2	7.6409	0.4114	0.5482
3	7.9358	0.1165	0.2533
4	8.0551	−0.0028	0.1340
6	8.1179	−0.0657	0.0711
8	8.1384	−0.0861	0.0507
12	8.1533	−0.1010	0.0358
16	8.1642	−0.1120	0.0248
24	8.1777	−0.1254	0.0114
32	8.1855	−0.1333	≈ 0

Table 10: Lag family on full `text8` at $V = 4,096$ (sequential members, all coding $x_{33}, \dots, x_n$; posterior 1.0 on $\delta = 1$; family = best member = 6.7132, again reproducing the (4096, 0) member of Table 5). $\hat{L} = 0.1368 \pm 0.0067$ is the learning-cost offset from the tail fit ($\delta \geq 4$) of Figure 1; entries ≈ 0 are consistent with zero within the fit’s residual. The *net* profile v crosses zero at $\delta \approx 4$: the range of lags over which a single lag repays its learning cost shrinks with V ($v > 0$ through $\delta = 8$ at $V = 1,024$), even though the information I is larger at every lag. This matches Tables 5 and 8: as V grows at fixed n , usable memory is confined to shorter distances.

itself: `gzip` 33,048,240 bytes = 2.64 bpc; `bzip2` 26,395,400 bytes = 2.11 bpc; `ppmd` 20,029,751 bytes = 1.60 bpc; `paq8h` 17,461,782 bytes = 1.40 bpc. The strongest current program of this kind, `cmix`, compresses `enwik8` (same length, but with markup and mixed case) to 14,623,723 bytes = 1.17 bpc. (ii) *The Hutter Prize* is a standing competition on `enwik9` only. The deliverable is a self-extracting archive; the measured quantity is its total size, decompressor included. The record is 114,156,155 bytes = 0.913 bpc (2023, a `cmix` derivative). Constraints: a single machine, less than 10 GB of RAM, less than 50 hours of running time, and a minimum improvement of 1% over the record, paid at about 5,000 € per percent. (iii) *Neural character-level models* are evaluated differently: `text8` is split into 90M/5M/5M characters for training, validation, and test; the model is trained on the training portion with as many passes as desired; the reported number is the bits per character the trained model assigns to the test portion. The cost of training is not part of the reported number, so it is not the length of any code for the file, and it is smaller than what the same model would attain in settings (i) or (ii). Reported values range from 1.19 (mLSTM) to 1.04 (Transformer-XL with dynamic evaluation). If we want a number comparable to these, we can report the bits our sequential code assigns to the last 5% of the token stream given the first 95%; this is available from any of our runs.

Our numbers in these units. Our best reduced-stream results convert as follows: the context-tree family at $V = 1,024$, 4.9507 bits/token, corresponds to 0.84 bits per character; at $V = 4,096$, 6.6807 bits/token corresponds to 1.14. Both omit the rare-word spellings described above. A first estimate of the missing part: encoding the spelling of every `<unk>` occurrence with a character model at 2–3 bits per character costs, at $V = 4,096$, 1.507 unknown-characters per token $\times$ (2–3) $\approx$ 3.0–4.5 bits/token. The total, 9.7–11.2 bits/token, corresponds to 1.65–1.90 bpc: between `bzip2` (2.11) and `ppmd` (1.60) on Mahoney’s list.

	δ	bits/token	$v(\delta)$	$I(\delta) = v(\delta) + \hat{L}$
0	(memoryless)	6.0636	—	—
	1	5.0564	1.0072	1.0224
	2	5.6962	0.3675	0.3827
	4	5.9902	0.0735	0.0887
	8	6.0418	0.0218	0.0371
	12	6.0488	0.0149	0.0301
	16	6.0544	0.0092	0.0244
	24	6.0632	0.0004	0.0157
	32	6.0673	−0.0037	0.0116
	48	6.0746	−0.0110	≈ 0
	64	6.0788	−0.0152	≈ 0

Table 11: Lag-family tail extension on full `text8` at $V = 1,024$ (all members coding $x_{65}, \dots, x_n$; posterior 1.0 on $\delta = 1$; family = best member = 5.0564). $\hat{L} = 0.0153 \pm 0.0052$ from the tail fit ($\delta \geq 4$) of Figure 1; entries ≈ 0 are consistent with zero within the fit residual. Information remains resolvable out to $\delta = 32$; the net-value horizon (where v crosses zero) sits at $\delta \approx 24$, against $\delta \approx 4$ at $V = 4,096$.

The spelling cost, measured. The per-occurrence estimate above can be replaced by a measurement of the construction we actually intend: spell each distinct word once, at its first occurrence, and treat it afterwards as an ordinary token (the escape mechanism of PPM, in sequential form; `scripts/spelling_experiment.py`). The character sequence this scheme transmits is the concatenation, in first-occurrence order, of the 253,854 distinct words plus a terminator each, 2,237,382 characters over a 27-symbol alphabet. Coding this stream with the context-tree family over character contexts of depth ≤ 3 costs 3.4312 bits/character (fixed depths: 4.284, 3.836, 3.596, 3.454) — more than the 2–3 assumed above, as the stream consists of rare words, largely names and foreign words, and the character model pays its own learning cost on only 2.2 million characters. The total is 7.68 Mbit = 0.4514 bits/token. A second term goes the other way. The fixed-alphabet token code (Section 3, $d = 2^{18}$) pays, at each first occurrence, for the identity of one *specific* previously unseen symbol; with spellings transmitted, the decoder only needs the event “a new symbol”, and since the exchangeable mixture puts equal mass on all unseen symbols, the saving at a first occurrence with u unseen symbols is $\log_2 u$. Summed over the corpus this is 4.24 Mbit = 0.2496 bits/token. The net cost of full fidelity is therefore $0.4514 - 0.2496 = 0.2018$ bits/token, and any token-level codelength X converts to a full-fidelity figure of $(X + 0.202)/5.8806$ bits/character. For the memoryless code, $X = 10.895$ gives

1.8870 bits/character,

the first number in this paper that is the length of a code for the file. It is below `bzip2` (2.11) and above `ppmd` (1.60), and uses no memory at all.

Prospects. With the conversion $(X + 0.202)/5.8806$ in hand, the remaining question is how far the token-level codelength X can be reduced, and the reference points translate into targets for X : matching `ppmd` (1.60) requires $X = 9.21$ bits/token at the full vocabulary; matching `paq8h` (1.40) requires $X = 8.03$. For scale: the memoryless code has $X = 10.895$, and the in-sample first-order conditional entropy of the full-vocabulary stream is 7.286 (Section 3), so a first-order sequential code beats `ppmd` if its learning cost stays below 1.9 bits/token and `paq8h` if it stays below

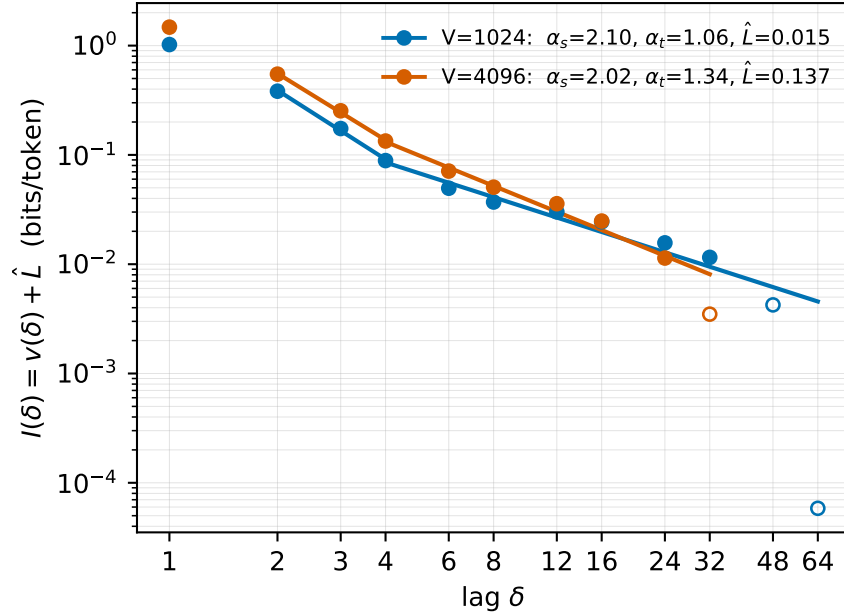


Figure 1: Corrected distance profiles $I(\delta) = v(\delta) + \hat{L}$ at both scales on log-log axes ($V=1,024$ merged over both runs), with the two-regime fits: short-range slope α_s fit on $2 \leq \delta \leq 4$, tail slope α_t and offset $\hat{L}$ fit on $\delta \geq 4$. Lag-1 points shown but excluded from fits; open circles mark corrected values below twice the fit residual (consistent with zero). The break at $\delta \approx 4$ separates the two regimes.

0.75. Whether it does is a measurement for the cluster (the reduced-stream trend, 98.6% $\rightarrow$ 78.2% capture over $V = 256 \rightarrow 4,096$, does not settle it). Two further reductions follow the two regimes of Figure 1: deeper adaptive context trees for the short-range regime ($\alpha_s \approx 2$), and the pooled lag predictors of scheme 5 for the tail ($\alpha_t \approx 1$). The programs between 1.60 and 1.1 on Mahoney’s list differ from `ppmd` mainly in that they combine predictions from many context sets, including non-contiguous ones, through mixing networks whose weights are set by online gradient methods; the pooled-lag construction computes a combination of the same kind, with weights determined by a posterior. Values below 1.0 bpc appear on `enwik9` (0.913) and in the neural setting (1.04 with training cost excluded); neither is within reach of the constructions in this paper as they stand.

Roadmap. The work is organized in four phases. *Phase 0* (done, this section): the full-fidelity accounting — escape mechanism, spelling model, index correction — giving the conversion above and the 1.8870 bpc memoryless figure. *Phase 1* (compute-bound; the code exists): the state-family and context-tree experiments at $V = 16,384, 65,536$, and the full vocabulary; profile evaluations are independent, so the work parallelizes over cluster nodes with a shared moment-table cache. Alongside: the KT-leaf version of the context tree (same tree, KT estimators in place of layered mixtures), which isolates the contribution of the layered prior, and depth 3. Expected outcome: a full-fidelity first-order and depth-2 figure; the targets for X above say what the learning cost must be for these to pass `ppmd` and `paq8h`. *Phase 2* (code, then compute): the sequential pooled evaluator — per-step predictive distributions from checkpointed expert tables, weight grids mixed at sequence level, context-dependent weights, doubling distance bands — with its test suite (exact enumeration

on a small alphabet; a one-hot weight recovering the lag-1 member; the cost of checkpoint staleness measured directly). Its scientific output is the marginal gain per added lag compared with $I(\delta)$, i.e. how much of the tail survives the overlap between lags. *Phase 3*: one sequential code containing all parts — spelling, full-vocabulary adaptive tree, gated lag pool — reported both as whole-file bits/character (comparable to Mahoney’s table) and as bits on the last 5% given the first 95% (comparable to the neural results).

7 Outlook: generating state maps, not designing them

This section records the candidate schemes for state maps that summarize *much longer* contexts, together with the criterion that selects them. Throughout, two design principles are held fixed: (a) every predictor is a Bayesian mixture; (b) the prior over state maps is *generated* by products or recursion — so that a broad range of memory complexities arises implicitly, as the layered construction does for distribution complexity — rather than designed explicitly. One fact keeps everything computable: any state map σ that is deterministic given the past and independent of the data leaves the successors of each state exchangeable, so per-state layered mixtures, profile-only evaluation, and deduplication apply unchanged, and the codes remain sequential. Five schemes, in increasing generality:

1. *Recursive tree growth (context trees with layered leaves)*. Generate σ by a branching process: each context splits into its per-token refinements with probability $\frac{1}{2}$, recursively, to maximal depth D . This recursive coin flip defines a prior over all variable-length suffix state maps, and the mixture over the entire family is computable without enumeration by the context-tree identity $\beta(\text{node}) = \frac{1}{2} q^{\text{avg}}(\text{node}) + \frac{1}{2} \prod_{\text{children}} \beta(\text{child})$, evaluated bottom-up — context-tree weighting with the layered mixture in place of KT at every node (the layered prior dominates KT on large alphabets [1]). The growth process is to state maps what coordinatewise multiplication is to distributions. (Implemented and running; see Section 5.2.)
2. *Random recursive automata (reservoir states)*. To reach beyond bounded suffixes: $s_t = h(s_{t-1}, x_t)$ with h a *random* transition function on S states — a state that can depend on the entire past. The prior is the randomness of h (the architecture-as-prior view of [1] in its purest form); the complexity parameter is S , and a mixture over an ensemble of sizes covers a range of complexities at negligible cost. The scheme measures what an undesigned recursive summary captures per state, compared with suffix states at equal S .
3. *Learned coarsenings (approximate causal states)*. Merge (state, token) pairs with similar continuation statistics — an information-bottleneck construction. Here σ depends on the data, so the code must be made sequential again: either two-part (transmit the merge structure at its description cost) or causal (cluster on a prefix, apply to the remainder). Comparing it with schemes 1–2 measures the value of designing states over generating them.
4. *Composed random coarsenings (the layered construction for memory itself)*. $\sigma = \varphi_R \circ \dots \circ \varphi_1$ with each φ_r a random elementary coarse-graining of (state $\times$ token); the composition depth R is the memory analogue of the emission depth L . By the mechanism that makes products of simplex draws spread their spectra, composition of random maps should spread the effective memory of typical draws, so that averaging over R yields one prior covering all memory complexities at logarithmic cost. Scheme 2 is the $R = 1$ special case. Compositions of random maps are analyzable, so this is the candidate for deriving — not merely fitting — the regret spectrum and scaling laws of memory.

5. *Pooled lag experts (combining predictions, not states)*. Every scheme above combines memories by *refining the state partition*, and Table 5 shows the price: refinement multiplies the number of states, so the learning cost grows multiplicatively. The complementary move is to combine *predictions*. Each lag δ defines a state map $\sigma^{(\delta)} : s_t = x_{t-\delta}$ — deterministic and data-independent, so the machinery of this paper applies unchanged, at cost *additive* in the number of lags ($\Delta \cdot V$ states, not V^Δ). The question is how to combine the per-lag estimates $p_\delta(x | x_{t-\delta})$ into one predictive distribution; three combinations of increasing strength follow. (a) The Bayesian mixture over δ (eq. (8) again) adds no cost and remains sequential, but can only *select*; its posterior measures the value of each lag. With pairwise information only and tractability constraints this is also what the optimal (Chow–Liu) tree structure degenerates to: attach x_t to its single best lag by mutual information. (b) Under the (naive-Bayes) assumption that the past tokens are conditionally independent given x_t , Bayes’ rule gives the logarithmic pool

$$q(x | \{x_{t-\delta}\}) \propto p(x) \prod_{\delta} \frac{p_\delta(x | x_{t-\delta})}{p(x)}, \quad \text{i.e.} \quad \log q = \log p(x) + \sum_{\delta} \text{PMI}(x; x_{t-\delta}) + \text{const} :$$

each lag contributes its pointwise mutual information with the candidate token and the prior is counted once. Unlike a mixture, the product *accumulates* evidence: two weak experts that agree yield a confident prediction, which no mixture can produce. Its failure mode is double counting when lags are redundant, and in text they are (Table 9, item (iii)). (c) The correction is the maximum-entropy joint consistent with all known pairwise marginals *including those among the conditioning variables* — a pairwise Markov random field $q(x | \cdot) \propto \exp(\sum_{\delta} \lambda_{\delta}(x, x_{t-\delta}))$, of which (b) is the special case of mutually independent lags. Its cheap surrogate is the weighted pool $q \propto p(x)^{1-\sum_{\delta} w_{\delta}} \prod_{\delta} p_{\delta}(x | x_{t-\delta})^{w_{\delta}}$, where $w_{\delta} = 1$ recovers (b) and $w_{\delta} < 1$ discounts redundancy. In our framework the weights are not computed from a dependence analysis but *learned*: a prior over w , mixed online, possibly with different weights in different contexts, at the usual $(\log_2 |\text{grid}|)/n$ price. Recursion — pool lags 1, 2 into one expert, lags 3, 4 into another, then pool the pools over doubling distance bands — reaches distance 2^R at cost linear in R . Two limits apply. First, pooling needs per-step predictive distributions and per-step normalization, so its implementation requires the sequential machinery deferred in Section 4. Second, pairwise probabilities cannot detect *synergy*: for a source in which x_t is a function of (x_{t-1}, x_{t-2}) jointly but independent of each alone, every pairwise conditional is flat and no pooling, however weighted, can predict; only combined states (schemes 1 and 4, or the product grids) capture such structure. Pooling and state refinement are therefore complements: pooling captures the *additive* part of the past at linear cost, while state refinement, at multiplicative cost, is the only way to capture the *synergistic* part. In these terms, the 57 split contexts of Table 7 are where the synergistic part is concentrated. The construction parallels attention in neural networks, where heads read tokens at several offsets and combine them with content-dependent weights. (Combination (a) is implemented and measured; see Section 5.3.)

Ultimate goal. The ideal, not directly learnable, is the minimal sufficient statistic of the past (causal states: two pasts equivalent iff they induce the same law of the future). The operational goal is the lift of [1] from i.i.d. sources to sources with memory: a single coherent Bayesian predictor, generated by layered and recursive randomness rather than designed priors, whose prior mass spans predictors of all joint (memory $\times$ distribution) complexities — so that every source is charged roughly its own complexity, with regret governed by identifiable discovery rates and scaling laws that can be derived. An exactly solvable instance of architectures-as-priors in which *recurrence*, not only depth, is part of the architecture; two nested layered constructions, one spreading emission complexity (L), one spreading memory complexity (R), each averaged, each priced, jointly adapting.

References

- [1] *A Layered Simplex Architecture for Large Alphabets*, 2026. Companion paper and repository (`product_model`).